

Implement Client Script & UI Policy (Incident)

Phase 5 - Project Development Phase — Incident

The **Project Development Phase** is where the approved design and plan are converted into actual ServiceNow configuration. For the Incident form, this means creating the required **UI Policies and Client Scripts**, configuring their conditions, and performing developer-level testing.

1. Development Scope

The development will focus on:

- Incident UI Policies
- Incident Client Scripts
- Field-level behavior
- Dynamic form behavior
- Client-side validation
- Error/information messages
- Basic developer testing

The principle is **UI Policy first, Client Script only when necessary**.

2. Development Task 1 — Create UI Policy

UI Policy: Resolution Fields

Table:

Incident [incident]

Condition:

State is Resolved

UI Policy Actions

Configure:

Field	Mandatory	Visible	Read-only
Resolution code	True	As required	False
Resolution notes	True	As required	False

Enable **Reverse if false** when the fields should return to their normal state after the Incident leaves the Resolved state.

Expected behavior

State = Resolved



Resolution code → Mandatory

Resolution notes → Mandatory

When the condition becomes false, the UI Policy reverses the applicable UI changes.

3. Development Task 2 — Category/Subcategory Behavior

If the approved requirement specifies dependent behavior between Category and Subcategory, implement the simplest supported solution.

For simple visibility/mandatory behavior:

Category selected



UI Policy



Subcategory becomes visible/mandatory

For more complex dynamic behavior:

Category changes



onChange Client Script



Evaluate new Category



Update dependent fields

An important development rule is to **avoid hard-coding choice values unless they have been verified in the target instance.**

4. Development Task 3 — Create onLoad Client Script

Create an **onLoad Client Script** only if the approved requirements require initialization logic.

Purpose

Ensure an existing Incident behaves correctly when the form is opened.

Conceptually:

```
function onLoad() {  
    // Initialize required client-side behavior.  
}
```

If a UI Policy already handles the required behavior, don't reproduce the same logic here.

Expected result

Opening an existing Incident should produce the same appropriate UI state as reaching that state through normal form interaction.

5. Development Task 4 — Create onChange Client Script

For dynamic behavior based on Category:

Type: onChange

Field: Category

Example structure:

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading) {  
        return;  
    }  
  
    if (!newValue) {  
        // Handle Category being cleared.  
        return;  
    }  
  
    // Apply approved Category-dependent behavior.  
}
```

Development considerations

The script should:

- Ignore unnecessary execution during form loading.
 - Handle the field being cleared.
 - Handle changing from one value to another.
 - Avoid leaving stale dependent values.
 - Avoid duplicating UI Policy functionality.
-

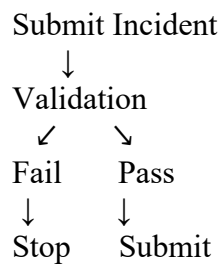
6. Development Task 5 — Create onSubmit Client Script

Use an **onSubmit Client Script** for requirements that need custom validation immediately before submission.

Example structure:

```
function onSubmit() {  
    // Perform approved validation.  
  
    if (/* validation fails */) {  
        g_form.addErrorMessage(  
            'Please complete the required Incident information.'  
        );  
  
        return false;  
    }  
  
    return true;  
}
```

Behavior



Client-side validation improves the user experience, but it should **not be treated as a security mechanism**. Rules that must always be enforced should also be handled appropriately on the server side.

7. Development Naming Convention

Use descriptive names.

UI Policy

INC - Resolved - Resolution Fields Mandatory

Client Scripts

INC - Category - Dependent Field Behavior

INC - Incident - Form Initialization

INC - Incident - Submission Validation

The exact naming convention should follow the organization's existing ServiceNow standards.

8. Developer Testing

After each component is created, test it independently.

UI Policy testing

1. Open Incident.
2. Set State = Resolved.
3. Verify Resolution code is mandatory.
4. Verify Resolution notes are mandatory.
5. Change State away from Resolved.
6. Verify the UI returns to the expected state.

onChange testing

1. Open Incident.
2. Change Category.
3. Verify dependent behavior.
4. Change Category again.
5. Verify previous dependent values do not cause inconsistent data.
6. Clear Category.
7. Verify the empty-value behavior.

onSubmit testing

1. Enter incomplete data.
 2. Submit.
 3. Verify submission is blocked when appropriate.
 4. Complete the required information.
 5. Submit again.
 6. Verify the Incident saves successfully.
-

9. Development Checklist

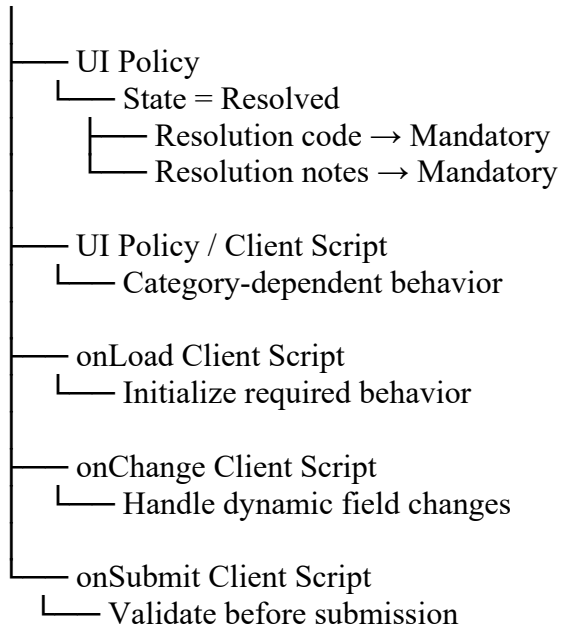
- ☐ Confirm correct Incident table.
- ☐ Verify actual field names.
- ☐ Verify actual choice values.
- ☐ Create UI Policies.
- ☐ Configure UI Policy Actions.
- ☐ Configure **Reverse if false** where appropriate.
- ☐ Create only required Client Scripts.

- ☐ Add appropriate isLoading handling to onChange scripts.
- ☐ Validate both positive and negative scenarios.
- ☐ Check for conflicts with existing scripts/policies.
- ☐ Test new Incidents.
- ☐ Test existing Incidents.
- ☐ Test field changes in both directions.
- ☐ Document configuration.
- ☐ Prepare changes for the testing environment.

10. Development Deliverable

At the end of this phase, the Incident configuration should look conceptually like:

Incident Form



The completed development package should then move into the **Testing/QA Phase**, where the planned test cases, regression scenarios, and user acceptance criteria are executed.